



CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

Faculty of Information and Communication Technology

2.1. Convolutional Neural Network

2.2 Recurrent Neural Network

2.3 Long-Short Term Memory

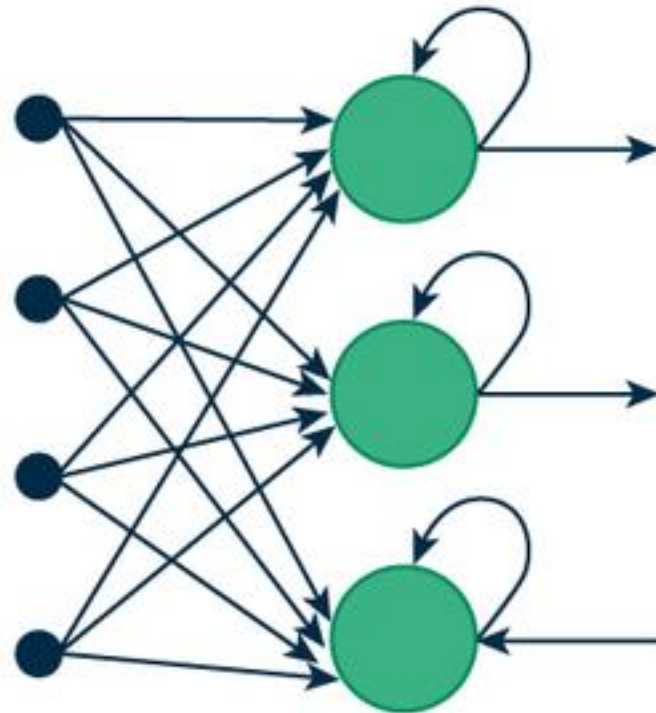
2.4 Optimization techniques

Recurrent Neural Network

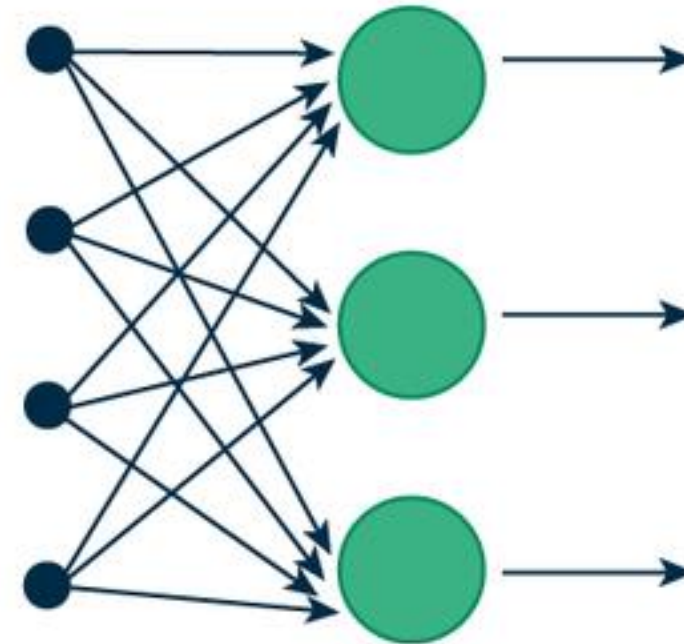
Recurrent Neural Network - RNN

- Designed to handle sequential data.
- Capable of remembering previous states through recurrent connections.
- RNNs use loops that allow information from previous steps to be fed back into the network.
- Suitable for problems with temporal continuity.

Comparison of RNNs and Traditional Neural Networks



(a) Recurrent Neural Network



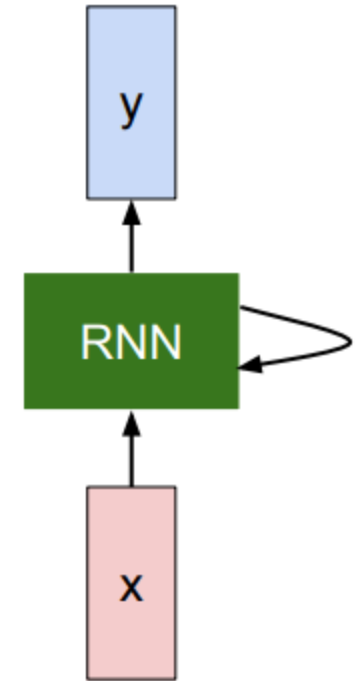
(b) Feed-Forward Neural Network

Structure of RNN

- Nodes (neurons) bind in a loop direction, where the output of one step of time is used as input to the next step.
- Each time t , the state is hidden h_t calculated based on current inputs x_t and the previous hidden state h_{t-1} :

$$h_t = fW(h_{t-1}, xt)$$

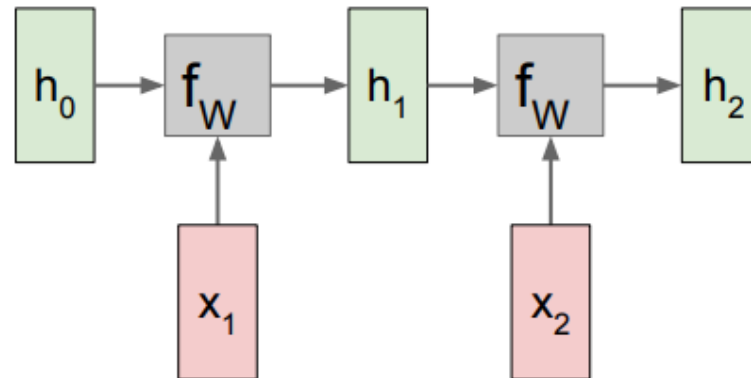
New Status Activation Function Old state Vector New Inputs



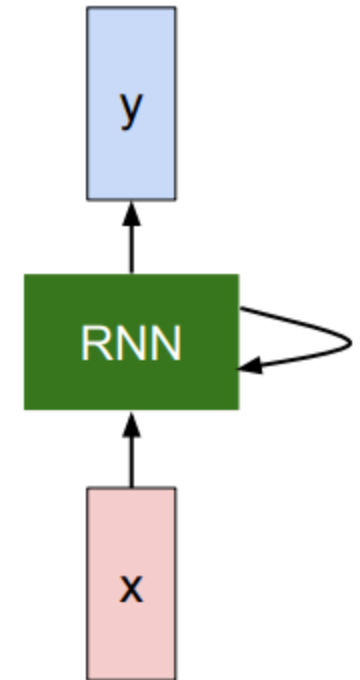
Recurrent Neurons

Structure of RNN

- String of inputs $x_0, x_1, \dots, x_t$ are events that occur in chronological order.
- Output y_t calculated from the current hidden state:



$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$
$$y_t = W_{hy} \cdot h_t$$



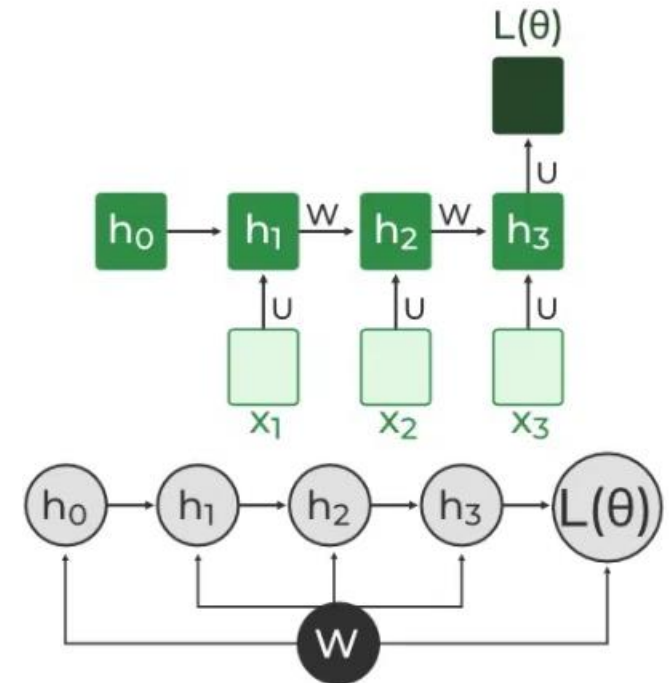
Recurrent Neurons

Where W_{hh} is the weight matrix for the recurrent neuron, W_{xh} is the weight matrix for the input neuron

Backpropagation Through Time - BPTT

In training process, RNN apply BPTT to update weight matrix based on the error of the chain data

- Loss function $L(\theta)$ depend on the state of the last hidden state h_3 , each hidden state relies on preceding ones, forming a sequential dependency chain:
- h_3 depend on h_2 and weight matrix W
- h_2 depend on h_1 and W
- h_1 depend on h_0 and W ,
- h_0 is the initial state, constant.



Backpropagation Through Time - BPTT

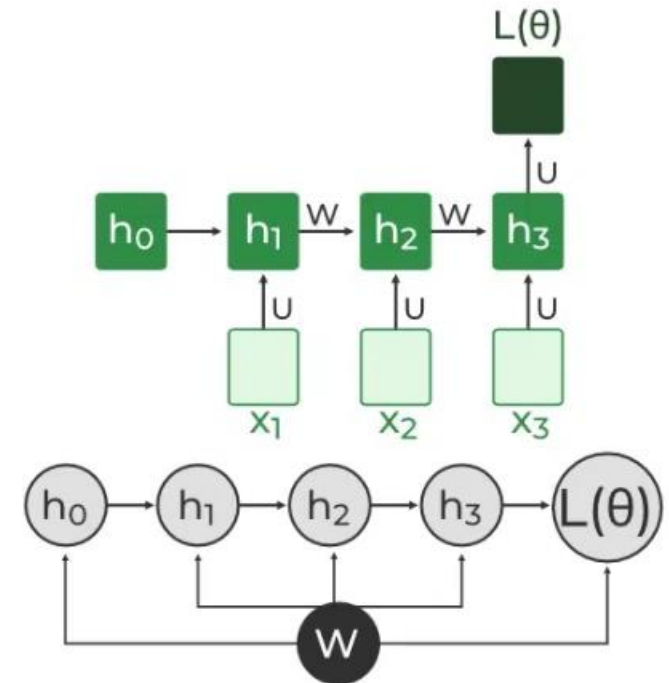
Reverse propagation application:

$$\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \frac{\partial h_3}{\partial W}$$

$$h_3 = \sigma(W \cdot h_2) + b$$

With σ is the activation function, $\frac{\partial h_3}{\partial W}$ depends on the previous hidden state:

$$\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \sum_{k=1}^3 \frac{\partial h_3}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}$$



Types of RNNs and Applications

One-to-One RNN

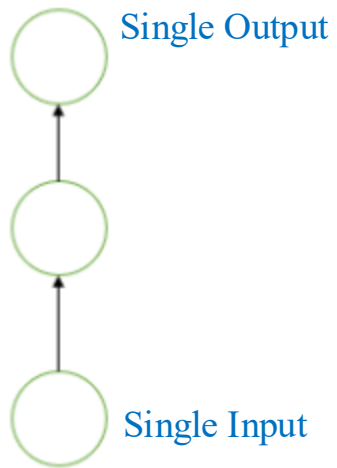


Image Classification

One-to-Many RNN

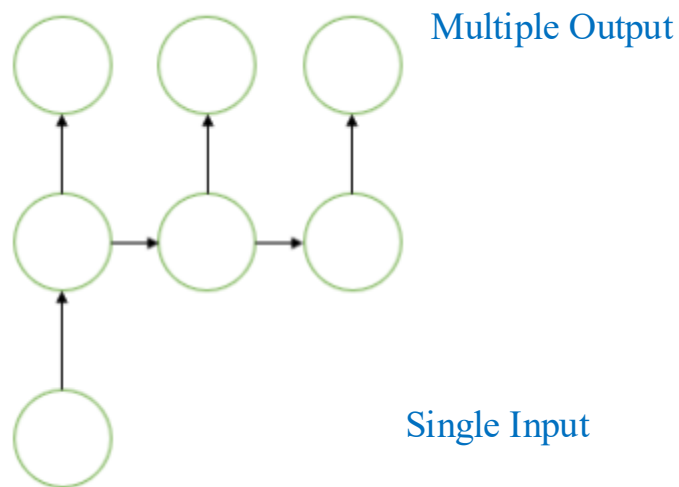
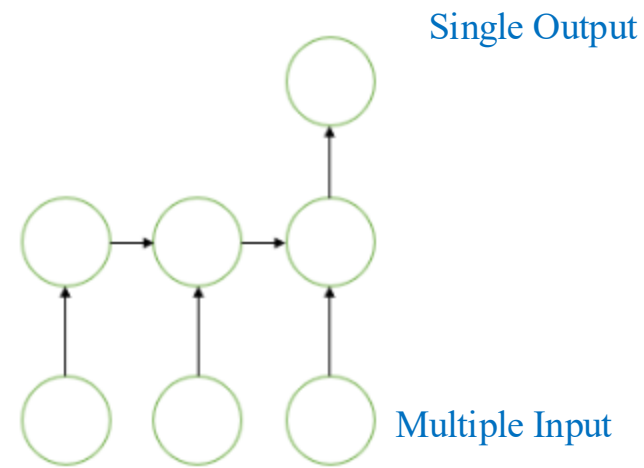


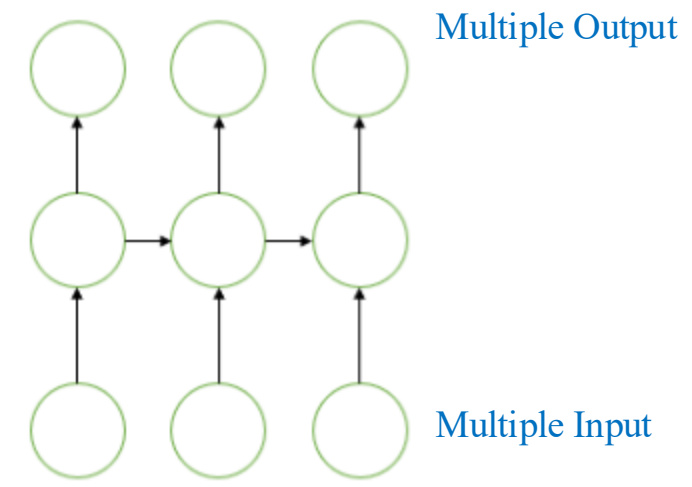
Image Captioning

One-to-Many RNN



Sentiment Analysis

Many-to-Many RNN



Translation

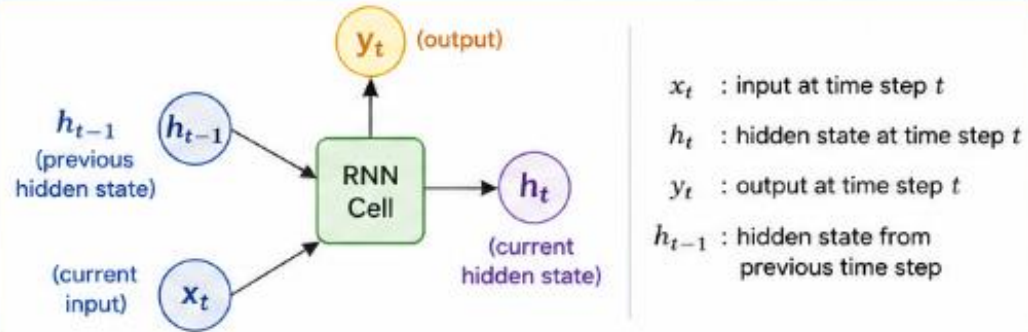
Comparison between CNN and RNN

Critical	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)
Data Types Processed:	Spatial Data (Images, Videos):	Sequential Data (Time Series, Text, Audio)
Network Structure	feedforward	Has loops to process sequential data
How to calculate the gradient:	Backpropagation	Backpropagation Through Time (BPTT)
The length of output/input	Fixed Input and Output Sizes	Can handle variable-length data
Most application	Image Recognition, Video Processing	Natural Language Processing (NLP), Speech Recognition

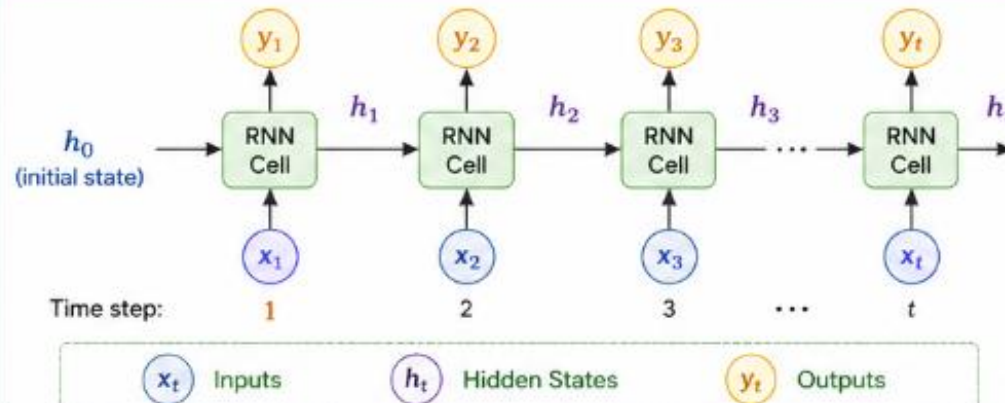
Advantages/Dis of RNN

Advantage	Disadvantage
Has the ability to handle inputs of any length.	Computation process is slow.
The model size does not increase with the size of the input.	Difficult to retrieve information from distant past.
Computation takes into account historical information.	Cannot consider future inputs for the current state.
Weights are shared over time.	Vanishing and Exploding Gradient problem

1. The RNN Cell



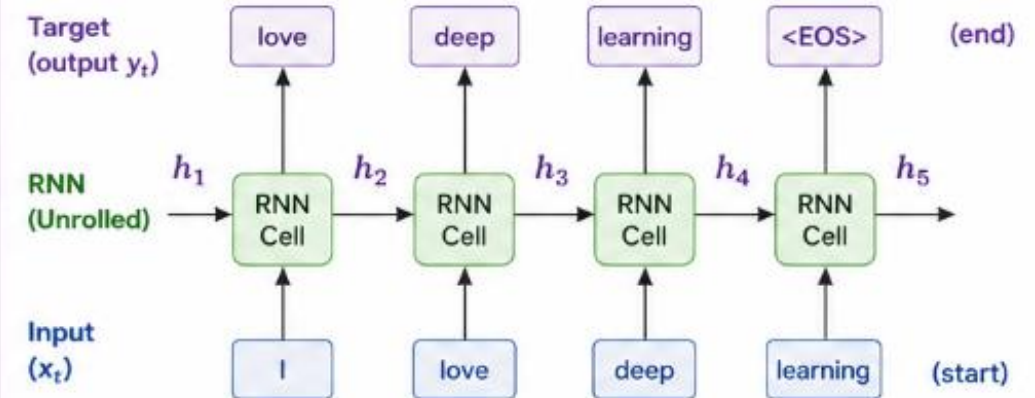
2. Unrolled RNN Over Time



3. Example: Next Word Prediction

Task: Given a sequence of words, predict the next word.

Example sequence: "I love deep learning <EOS>"



At each time step t , the RNN takes the current word (x_t) and the previous hidden state (h_{t-1}) to generate the next hidden state (h_t) and predict the next word (y_t).

4. Key Equations

Hidden state update:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

- W_{xh} : input-to-hidden weights
- W_{hh} : hidden-to-hidden (recurrent) weights
- b_h : bias for hidden state

Output computation:

$$y_t = \text{softmax}(W_{hy} h_t + b_y)$$

- W_{hy} : hidden-to-output weights
- b_y : bias for output
- $\text{softmax}()$: converts scores to probabilities

Where:

- x_t is the input vector at time t
- h_t is the hidden state (memory) at time t
- y_t is the output (e.g., next word probabilities) at time t

Example: Application of RNN in next word prediction.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

text = "This is CMC University, the first AI University in Vietnam"
chars = sorted(list(set(text)))
char_to_index = {char: i for i, char in enumerate(chars)}
index_to_char = {i: char for i, char in enumerate(chars)}
```


Example: Application of RNN in next word prediction.

```
seq_length = 3
sequences = []
labels = []

for i in range(len(text) - seq_length):
    seq = text[i:i + seq_length]
    label = text[i + seq_length]
    sequences.append([char_to_index[char] for char in seq])
    labels.append(char_to_index[label])

X = np.array(sequences)
y = np.array(labels)
```

Example: Application of RNN in next word prediction.

```
X_one_hot = tf.one_hot(X, len(chars))
y_one_hot = tf.one_hot(y, len(chars))

model = Sequential()
model.add(SimpleRNN(50, input_shape=(seq_length, len(chars)),
activation='relu'))
model.add(Dense(len(chars), activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X_one_hot, y_one_hot, epochs=100)

start_seq = "This is CMC"
generated_text = start_seq
```

Example: Application of RNN in next word prediction.

```
for i in range(50):
    x = np.array([[char_to_index[char] for char in generated_text[-
seq_length:]]])
    x_one_hot = tf.one_hot(x, len(chars))
    prediction = model.predict(x_one_hot)
    next_index = np.argmax(prediction)
    next_char = index_to_char[next_index]
    generated_text += next_char

print("Generated Text:")
print(generated_text)
```

```
Generated Text:
This is CMC University, the first AI University, the first AI
```